TravelMatch

Complete Developer Guide — Every File Explained

Django 4.2 | SQLite | Bootstrap 5.3 | Python 3.13 | AJAX Polling | Matching Algorithm

TABLE OF CONTENTS

1. Project Overview & Architecture
2. Project Configuration Files
3. Database Models — `core/models.py`
4. Views — `core/views.py`
5. Forms — `core/forms.py`
6. Matching Algorithm — `services/matching.py`
7. Settlement Algorithm — `services/settlement.py`
8. Templates — All HTML Pages
9. Static Files — CSS & JavaScript

10. How Everything Connects — End-to-End Flows

TravelMatch · Django Project · April 2026

CHAPTER 1

Project Overview & Architecture

What TravelMatch is, how the pieces fit together, and the full file structure.

What Is TravelMatch?

TravelMatch is a full-stack web platform that helps people find compatible travel companions. It has five core features:

- **Smart Matching** — scores users against each other using a weighted algorithm across budget, style, destinations, duration, and accommodation
- **Group Trip Management** — create or join trip groups using invite codes
- **Real-Time Chat** — AJAX polling fetches new messages every 2 seconds without refreshing
- **Collaborative Itinerary** — drag-to-reorder day plans shared across all members
- **Expense Tracking** — splits costs equally, exactly, or by percentage; auto-calculates who owes what

Technology Stack

LAYER	TECHNOLOGY	WHY
Backend framework	Django 4.2	Batteries-included: ORM, auth, admin, forms, CSRF all built in
Database	SQLite (file-based)	Zero setup for local development — single `db.sqlite3` file
Frontend CSS	Bootstrap 5.3.3	Responsive grid, ready-made components, dark/light theming
Frontend Icons	Bootstrap Icons 1.11	600+ icons, CSS-only
Drag-to-reorder	SortableJS 1.15	Touch-friendly drag-drop library, zero dependencies
Font	Inter (Google Fonts)	Clean, modern, highly readable
Real-time chat	AJAX long-polling	No WebSocket/Redis needed; works on basic hosting

Complete File Structure

```
travelmatch/ ← your project root
├── manage.py ← Django CLI (runserver, migrate, etc.)
├── requirements.txt ← Python package list (pip install -r requirements.txt)
├── db.sqlite3 ← the actual database file (do NOT commit to git)
│
├── travelmatch_project/ ← global config (settings, urls, wsgi)
│ ├── settings.py ← all configuration: DB, apps, static, auth
│ ├── urls.py ← master URL routing table
│ ├── asgi.py ← async server hook (deployment)
│ └── wsgi.py ← sync server hook (deployment)
│
├── core/ ← 100% of your application logic
│ ├── models.py ← database tables as Python classes
│ ├── views.py ← request handlers (URL → function → response)
│ ├── forms.py ← HTML form definitions & validation
│ ├── admin.py ← registers models in /admin panel
│ ├── apps.py ← app config boilerplate
│ ├── migrations/
│ │ └── 0001_initial.py← auto-generated SQL to create all tables
│ └── services/ ← pure business logic (no Django HTTP stuff)
```

```
│   ├── matching.py ← compatibility scoring algorithm
│   └── settlement.py ← debt simplification algorithm
│
├── templates/ ← all HTML pages
│   ├── base.html ← master layout (navbar, flash messages)
│   ├── home.html ← landing page for logged-out visitors
│   ├── dashboard.html ← home screen after login
│   ├── auth/ ← signup, onboarding, profile pages
│   ├── matches/ ← match feed (like/pass cards)
│   ├── groups/ ← group list, create, detail (chat/itinerary/expenses)
│   └── registration/ ← Django's built-in login page (overridden)
│
└── static/ ← CSS, JavaScript, images
    ├── css/ travelmatch.css ← design system, color overrides
    └── js/ travelmatch.js ← global JS (auto-dismiss alerts)
```

Django's Request Lifecycle

A browser sends a request → Django looks up the URL in `urls.py` → calls the matching function in `views.py` → that function reads/writes the database via `models.py` → renders an HTML template → sends the response back.

CHAPTER 2

Project Configuration Files

manage.py, settings.py, and urls.py — the three files that glue everything together.

`manage.py`

This is the command-line tool for your entire project. You *never edit it* — you only run it. Here's what the key commands do:

COMMAND	WHAT IT DOES
`python manage.py runserver`	Starts the development web server at http://127.0.0.1:8000
`python manage.py makemigrations`	Reads your models.py, detects changes, generates a migration file
`python manage.py migrate`	Applies all pending migrations to the database (creates/alters tables)
`python manage.py createsuperuser`	Creates an admin user for the /admin panel
`python manage.py shell`	Opens a Python REPL with Django fully loaded — great for testing

`travelmatch_project/settings.py`

The master configuration file. The most important settings:

```
# Which apps are active — Django's built-ins + your own
INSTALLED_APPS = [
    "django.contrib.admin",      # /admin panel
    "django.contrib.auth",       # login/logout/user model
    "django.contrib.sessions",   # cookie sessions
    "django.contrib.messages",   # flash messages
    "django.contrib.staticfiles",# CSS/JS serving
    "core",                      # YOUR application
]

# SQLite database — single file, zero setup
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}

# Where Django looks for HTML templates
TEMPLATES = [{ "DIRS": [BASE_DIR / "templates"], ... }]

# Where Django looks for CSS/JS/image files
STATICFILES_DIRS = [BASE_DIR / "static"]

# After login → go to dashboard. After logout → go to home.
LOGIN_REDIRECT_URL  = "dashboard"
LOGOUT_REDIRECT_URL = "home"
```

⚠ **DEBUG = True is only for development**

When `DEBUG = True`, Django shows you full error pages with stack traces — very useful for coding. But

never deploy to a public server with DEBUG = True

— it leaks code internals to anyone who visits.

`travelmatch_project/urls.py`

The **URL routing table**. This is the list of every URL your app responds to, paired with the Python function that handles it. Think of it like a switchboard.

```
from django.urls import include, path
from core import views as v

urlpatterns = [
    # Built-in Django auth URLs (login, password reset)
    path("accounts/", include("django.contrib.auth.urls")),
    path("accounts/signup/", v.signup_view, name="signup"),
    path("accounts/logout/", v.logout_view, name="logout"),

    # Core pages
    path("",            v.home,         name="home"),
    path("dashboard/",  v.dashboard,    name="dashboard"),
    path("onboarding/", v.onboarding,   name="onboarding"),
    path("profile/",    v.profile_view, name="profile"),

    # Matching
    path("matches/",                    v.match_feed,   name="match_feed"),
    path("matches/<int:user_id>/action/", v.match_action, name="match_action"),

    # Groups CRUD
    path("groups/",               v.group_list,   name="group_list"),
    path("groups/create/",        v.group_create, name="group_create"),
    path("groups/<int:pk>/",      v.group_detail, name="group_detail"),
    path("groups/<int:pk>/join/", v.group_join,   name="group_join"),
    ...
    # Chat, Itinerary, Expenses — all AJAX
    path("groups/<int:pk>/chat/messages/", v.chat_messages, name="chat_messages"),
]
```

The `name=` parameter lets templates reference URLs by name instead of hardcoding paths. For example, `{% url 'dashboard' %}` in a template automatically becomes `/dashboard/`.

The `<int:pk>` part is a **URL parameter** — it captures the number from the URL and passes it to the view function as `pk`. So `/groups/7/` calls `group_detail(request, pk=7)`.

CHAPTER 3

Database Models

core/models.py — Every database table as a Python class, and why each field exists.

`core/models.py`

In Django, you define your database structure in Python — not SQL. Each class is a table. Each class attribute is a column. Django's ORM translates your Python into SQL automatically. When you change a model and run `makemigrations`, Django figures out what SQL `ALTER TABLE` statements are needed.

💡 **Key concept: ForeignKey**

A `ForeignKey` means "this row belongs to one row in another table." For example, `GroupMember.group = ForeignKey(TripGroup)` means each membership record belongs to one group. `on_delete=models.CASCADE` means if that group is deleted, all its memberships are deleted too.

Model 1: Profile

Django's built-in `User` model only stores username, email, and password. `Profile` extends it with everything TravelMatch needs for matching.

```
class Profile(models.Model):
    # Links to exactly one User. If the User is deleted, Profile is deleted too.
    user          = models.OneToOneField(User, on_delete=models.CASCADE, related_name="profile")
    bio           = models.TextField(blank=True)
    avatar_url    = models.URLField(blank=True)

    # JSONField stores a Python list in a single DB column, e.g. ["europe", "east_asia"]
    destinations  = models.JSONField(default=list, blank=True)
    travel_months = models.JSONField(default=list, blank=True) # [6, 7, 8] = Jun/Jul/Aug

    # Single-value preferences — stored as short strings
    style         = models.CharField(max_length=20, choices=STYLE_CHOICES, default="mixed")
    budget        = models.CharField(max_length=10, choices=BUDGET_CHOICES, default="mid")
    duration      = models.CharField(max_length=20, choices=DURATION_CHOICES, default="week")
    accommodation = models.CharField(max_length=10, choices=ACCOMMODATION_CHOICES, default="hotel")

    # Flag: has the user completed the onboarding questionnaire?
    onboarding_complete = models.BooleanField(default=False)

    # @property = computed on the fly, not stored in DB
    @property
    def preference_dict(self):
        # Returns a plain dict — used by the matching algorithm
        return { "destinations": self.destinations, "style": self.style,
                 "budget": self.budget, "duration": self.duration,
                 "accommodation": self.accommodation }
```

Model 2: MatchAction

Records every like/pass decision a user makes. Used to prevent showing the same person twice.

```
class MatchAction(models.Model):
    actor  = models.ForeignKey(User, related_name="match_actions")  # who clicked
    target = models.ForeignKey(User, related_name="match_targets")  # who they acted on
    action = models.CharField(max_length=5, choices=[("like","Like"),("pass","Pass")])
    score  = models.FloatField(default=0.0)   # compatibility score saved at time of action

    class Meta:
        unique_together = ("actor", "target")  # can't act on same person twice
```

Model 3: TripGroup

```
class TripGroup(models.Model):
    name        = models.CharField(max_length=100)       # "Paris Spring 2026"
    destination = models.CharField(max_length=200)       # "Paris, France"
    start_date  = models.DateField(null=True, blank=True)
    end_date    = models.DateField(null=True, blank=True)
    owner       = models.ForeignKey(User, related_name="owned_groups")
    invite_code = models.CharField(max_length=16, unique=True, default=_gen_invite_code)
    max_members = models.PositiveSmallIntegerField(default=12)
    description = models.TextField(blank=True)

    @property
    def member_count(self):
        return self.memberships.count()  # counts GroupMember rows for this group
```

The invite code is generated by this helper — a random 10-character string from uppercase letters and digits:

```
def _gen_invite_code():
    alphabet = string.ascii_uppercase + string.digits
    return "".join(secrets.choice(alphabet) for _ in range(10))
# Example output: "AMEZ5644IS"
```

Model 4: GroupMember

The **join table** between users and groups — also stores the user's role within the group.

```
class GroupMember(models.Model):
    group     = models.ForeignKey(TripGroup, related_name="memberships")
    user      = models.ForeignKey(User,      related_name="group_memberships")
    role      = models.CharField(choices=[("owner",...),("member",...)], default="member")
    joined_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ("group", "user")  # can't be in same group twice
```

Model 5: Message

```
class Message(models.Model):
    group   = models.ForeignKey(TripGroup, related_name="messages")
    sender  = models.ForeignKey(User, related_name="messages")
    body    = models.TextField()
    sent_at = models.DateTimeField(auto_now_add=True)

    def to_dict(self):
        # Converts message to JSON-serializable dict for the chat API
        return { "id": self.pk, "sender_id": self.sender_id,
                 "sender_name": self.sender.username,
                 "body": self.body, "sent_at": self.sent_at.isoformat() }
```

Models 6 & 7: ItineraryDay and ItineraryActivity

Two-level hierarchy: a group has many days, each day has many activities.

ITINERARYDAY FIELD	PURPOSE
`group`	Which trip group this day belongs to
`day_number`	1, 2, 3… auto-incremented by the view
`date`	Optional specific calendar date
`title`	"Arrival Day", "Beach Day" etc.

ITINERARYACTIVITY FIELD	PURPOSE
`day`	Which ItineraryDay this belongs to
`name`	"Visit Eiffel Tower"
`start_time / end_time`	Optional time range for the activity
`location_name`	"Eiffel Tower, Paris"
`assignee`	Optional — which group member is responsible
`order`	Integer used for drag-to-reorder (0, 1, 2…)

Models 8, 9 & 10: Expense, ExpenseAllocation, BalanceLedger

These three work together to track shared costs:

MODEL	WHAT IT STORES	EXAMPLE
`Expense`	The expense itself — who paid, how much, what for	Alice paid $120 for hotel
`ExpenseAllocation`	Each member's individual share of that expense	Alice: $40, Bob: $40, Carol: $40
`BalanceLedger`	Running net balance per person (updated incrementally)	Alice: +$80, Bob: -$40, Carol: -$40

🔑 **Why BalanceLedger exists**

Without it, to show balances you'd have to sum all expense allocations every time — slow for large groups. Instead, every time an expense is added or deleted, we update the ledger instantly. Balance lookups then take a single fast database query (O(1) instead of O(n)).

How the ledger is updated when Alice pays $120 split equally among 3 people:

```
# Alice paid $120, split equally 3 ways ($40 each)
# delta = (amount_paid − own_share) for payer, (−own_share) for others

Alice:  delta = (120 − 40) = +80   # Alice is owed $80
Bob:    delta = (0   − 40) = −40   # Bob owes $40
Carol:  delta = (0   − 40) = −40   # Carol owes $40
```

CHAPTER 4

Views — Request Handlers

core/views.py — Every URL maps to a function here. This is where requests come in and responses go out.

A view function receives an HTTP `request` object and returns either an HTML response (from `render()`) or a JSON response (from `JsonResponse()`). Every view in this file corresponds to a URL in `urls.py`.

Authentication Views

home()

```
def home(request):
    if request.user.is_authenticated:
        return redirect("dashboard")  # already logged in → skip landing page
    return render(request, "home.html")
```

signup_view()

```
def signup_view(request):
    if request.method == "POST":
        form = SignUpForm(request.POST)
        if form.is_valid():
            user = form.save()               # creates User with hashed password
            Profile.objects.create(user=user) # create blank Profile for them
            login(request, user)              # log them in automatically
            return redirect("onboarding")  # → go fill in preferences
    else:
        form = SignUpForm()
    return render(request, "auth/signup.html", {"form": form})
```

> ℹ️ **GET vs POST pattern**
>
> All form views use the same pattern: if the browser sends GET (just opening the page), show the empty form. If it sends POST (submitted the form), validate and process. This is called the PRG (Post/Redirect/Get) pattern and prevents duplicate submissions on browser refresh.

The Onboarding View

This is the travel preference questionnaire shown once after signup. The tricky part is handling **JSONField lists** from checkboxes:

```
def onboarding(request):
    profile, _ = Profile.objects.get_or_create(user=request.user)
    if request.method == "POST":
        form = OnboardingForm(request.POST, instance=profile)
        if form.is_valid():
            p = form.save(commit=False)  # don't save to DB yet

            # Checkboxes send multiple values for the same field name
            # request.POST.getlist() collects ALL checked values as a list
            p.destinations   = request.POST.getlist("destinations")
            p.travel_months  = [int(m) for m in request.POST.getlist("travel_months")]

            p.onboarding_complete = True
            p.save()  # NOW save to DB
            return redirect("match_feed")
```

Match Feed View

```
def match_feed(request):
    profile, _ = Profile.objects.get_or_create(user=request.user)
    if not profile.onboarding_complete:
        return redirect("onboarding")  # force them to set up preferences first

    # get_ranked_matches returns list of (User, score) where score is 0.0–1.0
    # multiply by 100 so template shows "87%" instead of "0.87"
    ranked = [(u, round(s * 100, 1)) for u, s in get_ranked_matches(request.user, limit=20)]
    return render(request, "matches/match_feed.html", {"ranked": ranked})
```

Match Action View (AJAX)

```
@login_required
@require_POST  # only accept POST requests — reject GET
def match_action(request, user_id):
    action = request.POST.get("action")  # "like" or "pass"
    target = get_object_or_404(User, pk=user_id)

    # Compute score at time of action and save it
    score = compute_compatibility(profile.preference_dict, target_profile.preference_dict)

    MatchAction.objects.update_or_create(
        actor=request.user, target=target,
        defaults={"action": action, "score": score},
    )
    return JsonResponse({"ok": True})  # JavaScript receives this
```

Group Detail View

The most data-heavy view — loads everything needed for the group workspace:

```
def group_detail(request, pk):
    group      = get_object_or_404(TripGroup, pk=pk)
    membership = GroupMember.objects.filter(group=group, user=request.user).first()
    is_member  = membership is not None
    is_owner   = group.owner == request.user

    # prefetch_related avoids N+1 queries (loads all activities in 1 SQL query)
    itinerary_days = ItineraryDay.objects.filter(group=group)\
                        .prefetch_related("activities__assignee")
    expenses       = Expense.objects.filter(group=group).select_related("paid_by")
    balances       = {b.user: b.net_balance for b in
                      BalanceLedger.objects.filter(group=group).select_related("user")}
    settlement     = compute_group_settlement(group) if is_member else []

    return render(request, "groups/group_detail.html", {
        "group": group, "is_member": is_member, "is_owner": is_owner,
        "itinerary_days": itinerary_days, "expenses": expenses,
        "balances": balances, "settlement": settlement,
        "expense_form": ExpenseForm(group=group),
    })
```

Chat Views (AJAX Polling)

```
def chat_messages(request, pk):
    # JavaScript sends ?since=42 meaning "only give me messages newer than ID 42"
    since_id = int(request.GET.get("since", 0))
    msgs = Message.objects.filter(group=group, pk__gt=since_id)[:100]
    return JsonResponse({"messages": [m.to_dict() for m in msgs]})

def chat_send(request, pk):
    body = request.POST.get("body", "").strip()
    msg  = Message.objects.create(group=group, sender=request.user, body=body)
    return JsonResponse({"ok": True, "message": msg.to_dict()})
```

Expense Create View (Most Complex)

```
def expense_create(request, pk):
    member_ids = [m.user_id for m in GroupMember.objects.filter(group=group)]

    if split_type == "equal":
        share  = amount / len(member_ids)
        splits = {uid: share for uid in member_ids}

    # transaction.atomic() = all-or-nothing: if anything fails, nothing is saved
    with transaction.atomic():
        expense = Expense.objects.create(...)

        for uid, owed in splits.items():
            ExpenseAllocation.objects.create(expense=expense, user_id=uid, amount_owed=owed)

            # Update running balance for this member
            delta = (amount - owed) if uid == paid_by.pk else -owed
            ledger, _ = BalanceLedger.objects.get_or_create(group=group, user_id=uid,
                                                            defaults={"net_balance": 0})
            ledger.net_balance += delta
            ledger.save()

    return JsonResponse({"ok": True})
```

CHAPTER 5

Forms

core/forms.py — HTML form definitions, validation logic, and Bootstrap widget styling.

`core/forms.py`

Django forms have two jobs: **render HTML** (generate the input fields) and **validate input** (check data before saving). Every form in this file maps to either a model or a standalone process.

SignUpForm

```
class SignUpForm(UserCreationForm):
    # UserCreationForm already has username, password1, password2
    # We just add email on top
    email = forms.EmailField(required=True)

    class Meta:
        model  = User
        fields = ("username", "email", "password1", "password2")

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Apply Bootstrap's form-control class to every field automatically
        for field in self.fields.values():
            field.widget.attrs["class"] = "form-control"
```

OnboardingForm

The most interesting form — handles multi-select checkboxes for destinations and months:

```python
class OnboardingForm(forms.ModelForm):
    # MultipleChoiceField renders as a list of checkboxes
    destinations = forms.MultipleChoiceField(
        choices=DESTINATION_CHOICES,         # [("europe","Europe"), ...]
        widget=forms.CheckboxSelectMultiple,
        required=False,
    )
    travel_months = forms.MultipleChoiceField(
        choices=MONTH_CHOICES,               # [(1,"January"), (2,"February"), ...]
        widget=forms.CheckboxSelectMultiple,
        required=False,
    )

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        if self.instance and self.instance.pk:
            # Pre-tick the checkboxes that were previously saved
            self.initial["destinations"]  = self.instance.destinations or []
            self.initial["travel_months"] = [str(m) for m in (self.instance.travel_months or [])]
```

ActivityForm

The assignee dropdown is **dynamic** — it only shows members of the specific group:

```python
class ActivityForm(forms.ModelForm):
    def __init__(self, *args, group=None, **kwargs):
        super().__init__(*args, **kwargs)
        if group:
            # Get user IDs who are members of this group
            member_ids = GroupMember.objects.filter(group=group).values_list("user_id", flat=True)
            # Restrict assignee dropdown to only those users
            self.fields["assignee"].queryset = User.objects.filter(pk__in=member_ids)
            self.fields["assignee"].empty_label = "— No assignee —"
```

CHAPTER 6

Matching Algorithm

core/services/matching.py — The mathematical heart of TravelMatch. How compatibility is scored.

`core/services/matching.py`

This file answers: *"How similar are two users' travel preferences?"* The answer is a number between 0 and 1, where 1 means perfect match.

THE COMPATIBILITY FORMULA

```
Score(A, B) = (budget_sim × 0.35)
            + (destinations_sim × 0.25)
            + (style_sim × 0.20)
            + (duration_sim × 0.12)
            + (accommodation_sim × 0.08)
```

FEATURE	WEIGHT	METHOD	LOGIC
budget	35%	Ordinal similarity	Exact = 1.0 | One tier apart = 0.5 | Two tiers apart = 0.0
destinations	25%	Jaccard similarity	|intersection| ÷ |union| of their destination sets
style	20%	Custom	Exact = 1.0 | Either is "mixed" = 0.6 | Different = 0.0
duration	12%	Ordinal similarity	Same as budget — weekend/week/twoweeks/extended are ordered tiers
accommodation	8%	Exact match	Same preference = 1.0 | Different = 0.0

Ordinal Similarity

Used for budget and duration — these are ordered scales where "nearby" values are somewhat compatible:

```
# Budget tiers in order: economy → mid → luxury
BUDGET_TIERS   = ["economy", "mid", "luxury"]
DURATION_TIERS = ["weekend", "week", "twoweeks", "extended"]

def _ordinal_sim(val_a, val_b, tiers):
    dist = abs(tiers.index(val_a) - tiers.index(val_b))
    return 1.0 if dist == 0 else (0.5 if dist == 1 else 0.0)

# Examples:
# economy vs economy → dist=0 → 1.0 (perfect)
# economy vs mid     → dist=1 → 0.5 (partial)
# economy vs luxury  → dist=2 → 0.0 (no match)
```

Jaccard Similarity (Destinations)

Jaccard measures the overlap between two sets. It's ideal for destinations because two people might both want to visit 3 out of 5 of the same regions.

```
def _jaccard_sim(set_a, set_b):
    a, b = set(set_a), set(set_b)
    if not a and not b: return 0.0
    return len(a & b) / len(a | b)  # intersection ÷ union

# Example:
# Alice: ["europe", "east_asia", "caribbean"]
# Bob:   ["europe", "east_asia", "africa"]
# intersection = {"europe", "east_asia"} → size 2
# union        = {"europe","east_asia","caribbean","africa"} → size 4
# Jaccard      = 2/4 = 0.50
```

Worked Example

Alice (adventure, mid, week, hotel, europe+asia) vs Bob (cultural, mid, week, hotel, europe+americas):

```
budget_sim        = ordinal("mid", "mid")           = 1.00
destinations_sim  = jaccard(["europe","east_asia"],
                            ["europe","south_america"]) = 1/3 = 0.33
style_sim         = style("adventure", "cultural")  = 0.00
duration_sim      = ordinal("week", "week")          = 1.00
accommodation_sim = "hotel" == "hotel"               = 1.00

Score = (1.00×0.35) + (0.33×0.25) + (0.00×0.20) + (1.00×0.12) + (1.00×0.08)
      = 0.35 + 0.083 + 0.00 + 0.12 + 0.08
      = 0.633  →  63.3% compatible
```

get_ranked_matches()

```python
def get_ranked_matches(current_user, limit=20, threshold=0.0):
    # Step 1: find users already acted on (liked or passed)
    acted_ids = set(MatchAction.objects.filter(actor=current_user)
                                .values_list("target_id", flat=True))
    acted_ids.add(current_user.pk)  # exclude self too

    # Step 2: get candidates who completed onboarding
    candidates = User.objects.exclude(pk__in=acted_ids)\
                             .filter(profile__onboarding_complete=True)\
                             .select_related("profile")

    # Step 3: score each candidate
    results = []
    for user in candidates:
        score = compute_compatibility(my_pref, user.profile.preference_dict)
        if score >= threshold:
            results.append((user, score))

    # Step 4: sort descending by score, return top N
    results.sort(key=lambda x: x[1], reverse=True)
    return results[:limit]
```

CHAPTER 7

Settlement Algorithm

core/services/settlement.py — Minimize bank transfers to settle group debts.

`core/services/settlement.py`

After a trip, some people are owed money and some owe money. The naive solution is for everyone to pay everyone — which could be many transfers. This algorithm finds the **minimum number of transfers** to settle everything.



Mathematical guarantee

For a group of n members, this algorithm always produces at most **n - 1 transfers**, no matter how many expenses were added.

The Algorithm

```python
def compute_settlement(balances: dict) -> list:
    """
    Input:  {user_id: net_balance}
            positive balance = others owe this user
            negative balance = this user owes others
    Output: [{"from_user": id, "to_user": id, "amount": float}]
    """

    # Split into creditors (owed money) and debtors (owe money)
    creditors = [[uid,  bal] for uid, bal in balances.items() if bal >  EPSILON]
    debtors   = [[uid, -bal] for uid, bal in balances.items() if bal < -EPSILON]

    # Sort both by amount (largest first)
    creditors.sort(key=lambda x: x[1], reverse=True)
    debtors.sort(key=lambda x: x[1], reverse=True)

    transfers = []
    ci, di = 0, 0

    while ci < len(creditors) and di < len(debtors):
        cred_id, credit = creditors[ci]
        debt_id, debt   = debtors[di]

        # Transfer the smaller of what the creditor is owed vs what the debtor owes
        amount = min(credit, debt)
        transfers.append({"from_user": debt_id, "to_user": cred_id, "amount": round(amount, 2)})

        # Reduce both sides
        creditors[ci][1] -= amount
        debtors[di][1]   -= amount

        # If either reaches zero, move to the next person
        if creditors[ci][1] < EPSILON: ci += 1
        if debtors[di][1]   < EPSILON: di += 1

    return transfers
```

Step-by-Step Example

4 people went on a trip. After all expenses:

PERSON	NET BALANCE	MEANING
Alice	+$120	Others owe Alice $120
Bob	+$30	Others owe Bob $30
Carol	-$80	Carol owes $80
Dave	-$70	Dave owes $70

```
Step 1 Sort creditors: Alice($120), Bob($30)
          Sort debtors: Carol($80), Dave($70)

Step 2 Match Carol($80) → Alice($120): transfer $80
          Carol settled. Alice now needs $40 more.

Step 3 Match Dave($70) → Alice($40): transfer $40
          Alice settled. Dave still owes $30.

Step 4 Match Dave($30) → Bob($30): transfer $30
          Both settled. Done!

Result: 3 transfers instead of potentially 12 (4×3).
```

The Convenience Wrapper

```python
def compute_group_settlement(group):
    # Read balances from BalanceLedger for this group
    ledger   = BalanceLedger.objects.filter(group=group).select_related("user")
    balances = {entry.user_id: float(entry.net_balance) for entry in ledger}

    # Run the algorithm (returns user IDs)
    raw = compute_settlement(balances)

    # Look up actual User objects so templates can show usernames
    user_map = {u.pk: u for u in User.objects.filter(pk__in=balances.keys())}
    return [{"from_user": user_map[t["from_user"]],
             "to_user": user_map[t["to_user"]],
             "amount": t["amount"]} for t in raw]
```

CHAPTER 8

Templates — All HTML Pages

Every template file, what it shows, and how Django's template language works.

Django Template Language basics

`{{ variable }}` outputs a value. `{% tag %}` executes logic (if/for/url/block). `{% extends "base.html" %}` means "use base.html as the layout and fill in its blocks." `{% block content %}...{% endblock %}` is the section the child template fills in.

base.html — The Master Layout

Every other template extends this. It defines the page structure:

```
<!-- Loads {% load static %} so we can reference CSS/JS files -->
<head>
  <!-- Inter font, Bootstrap 5.3, Bootstrap Icons, SortableJS, travelmatch.css -->
  <link href="{% static 'css/travelmatch.css' %}" rel="stylesheet" />
</head>

<nav class="navbar tm-navbar sticky-top">
  <a href="{% url 'home' %}">TravelMatch</a>

  <!-- Active link detection: adds "active" class if current page matches -->
  <a class="nav-link {% if request.resolver_match.url_name == 'dashboard' %}active{% endif %}"
     href="{% url 'dashboard' %}">Dashboard</a>
</nav>

<main>
  <!-- Flash messages (success/error) from views.py messages.success() -->
  {% for message in messages %}
  <div class="alert alert-{{ message.tags }}">{{ message }}</div>
  {% endfor %}

  {% block content %}{% endblock %}  <!-- child templates fill this in -->
</main>

{% block extra_js %}{% endblock %}  <!-- per-page JavaScript goes here -->
```

home.html — Landing Page

Shown to logged-out visitors. Contains the hero section, 4 feature cards, and the "How it works" 3-step section. If the user is already logged in, the view redirects them to dashboard so this page is never shown to them.

registration/login.html — Sign In

The login form. Manually writes the username and password inputs with Bootstrap classes, because Django's built-in `AuthenticationForm` doesn't add Bootstrap classes automatically.

auth/signup.html — Sign Up

Uses a for loop over form fields to render each input with its label and any validation errors:

```
{% for field in form %}
<div class="mb-3">
  <label>{{ field.label }}</label>
  {{ field }}  <!-- renders the actual <input> element -->
  {% if field.errors %}
  <div class="invalid-feedback d-block">{{ field.errors|join:", " }}</div>
  {% endif %}
</div>
{% endfor %}
```

auth/onboarding.html and auth/profile.html

The travel preference questionnaire. Shows checkboxes for destinations (10 regions) and travel months (12 months). The tricky part is pre-ticking boxes when editing existing preferences:

```
<!-- Checkbox ticked if this value exists in the user's saved list -->
<input type="checkbox" name="destinations" value="{{ value }}"
  {% if value in form.initial.destinations %}checked{% endif %}>
```

dashboard.html

Two-column layout after login. Left column shows group cards; right column shows the top 3 compatibility matches with progress bars:

```
<!-- Progress bar width = compatibility score (0–100) -->
<div class="progress-bar" style="width:{{ score|floatformat:0 }}%"></div>
<span>{{ score|floatformat:0 }}%</span>
```

matches/match_feed.html

The discover page. Each match renders as a card. Like/Pass buttons use JavaScript to AJAX-submit and animate the card away without a page reload:

```
// JavaScript in the template
document.querySelectorAll('.match-action-btn').forEach(btn => {
  btn.addEventListener('click', function() {
    const fd = new FormData();
    fd.append('action', this.dataset.action);  // "like" or "pass"
    fd.append('csrfmiddlewaretoken', csrfToken);

    fetch(`/matches/${userId}/action/`, {method: 'POST', body: fd})
      .then(r => r.json())
      .then(data => {
        if (data.ok) card.style.opacity = '0';  // fade out
        setTimeout(() => card.remove(), 300);    // remove after animation
      });
  });
});
```

groups/group_detail.html — The Big One

This template has 4 Bootstrap tabs: Chat, Itinerary, Expenses, Members.

Chat Tab

Shows past messages on load. JavaScript polls every 2 seconds for new ones:

```
let lastId = 0;  // tracks last message ID seen

async function fetchMessages() {
  const r    = await fetch(`/groups/${groupPk}/chat/messages/?since=${lastId}`);
  const data = await r.json();
  data.messages.forEach(msg => {
    lastId = Math.max(lastId, msg.id);  // update cursor
    chatBox.appendChild(renderMessage(msg, isSelf));
  });
}
setInterval(fetchMessages, 2000);  // poll every 2 seconds
```

Itinerary Tab

Day cards with sortable activity lists. SortableJS handles the drag-drop and calls the server to save the new order:

```
new Sortable(list, {
  animation: 150,
  handle: '.bi-grip-vertical',  // only drag by the grip icon
  onEnd: async () => {
    const ids = [...list.querySelectorAll('[data-activity-id]')]
                  .map(el => parseInt(el.dataset.activityId));
    // Send new order to server
    await fetch(`/groups/${groupPk}/itinerary/reorder/`, {
      method: 'POST',
      headers: {'Content-Type': 'application/json', 'X-CSRFToken': csrfToken},
      body: JSON.stringify({ordered_ids: ids}),
    });
  },
});
```

Expenses Tab

Shows expense list, per-member balances (green if owed, red if owing), and the settlement plan showing minimum transfers needed.

CHAPTER 9

Static Files — CSS & JavaScript

travelmatch.css and travelmatch.js — The design system and global behaviour.

`static/css/travelmatch.css`

This file **customizes Bootstrap** to use TravelMatch's indigo/violet color palette and adds project-specific components. It uses CSS custom properties (variables) so you can change the primary color in one place.

```css
/* Override Bootstrap's default blue with indigo-violet */
:root {
  --tm-primary:     #6366f1;   /* main indigo */
  --tm-primary-dark:#4f46e5;   /* darker for hover states */
  --bs-primary:     #6366f1;   /* Bootstrap reads this variable! */
  --bs-primary-rgb: 99, 102, 241;
}

/* Recolor all .btn-primary buttons */
.btn-primary {
  --bs-btn-bg:       #6366f1;
  --bs-btn-hover-bg: #4f46e5;
  font-weight: 500;
  border-radius: .5rem;
}

/* Active nav links get an indigo highlight */
.tm-navbar .nav-link:hover,
.tm-navbar .nav-link.active {
  background: rgba(99,102,241,.08);
  color: var(--tm-primary);
}

/* Active tab gets an indigo underline */
.nav-tabs .nav-link.active {
  color: var(--tm-primary);
  border-bottom: 2px solid var(--tm-primary);
}

/* Drag-drop ghost element (while dragging) */
.sortable-ghost {
  opacity: .4;
  background: #e0e7ff;  /* light indigo tint */
}
```

`static/js/travelmatch.js`

Intentionally minimal — only contains global behavior that applies to every page. Per-page logic (chat polling, drag-drop, expense forms) lives in `{% block extra_js %}` within each template to keep related code together.

```
/* Auto-dismiss flash messages after 5 seconds */
document.addEventListener('DOMContentLoaded', function() {
  document.querySelectorAll('.alert.alert-dismissible').forEach(function(el) {
    setTimeout(function() {
      const bsAlert = bootstrap.Alert.getOrCreateInstance(el);
      bsAlert.close();
    }, 5000);  // 5000ms = 5 seconds
  });
});
```

Bootstrap's `Alert.close()` method triggers the CSS fade-out animation before removing the element — so the alert doesn't just disappear abruptly.

💡 **CSRF tokens in AJAX requests**

Every POST request in Django must include a CSRF token to prevent cross-site request forgery attacks. In the templates, we include a hidden form `<form id="csrf-form">{% csrf_token %}</form>` and JavaScript reads the token from it: `document.querySelector('#csrf-form [name=csrfmiddlewaretoken]').value`. This is appended to every AJAX POST.

CHAPTER 10

How Everything Connects

End-to-end flows showing how all the files work together for each major feature.

Flow 1: User Signs Up

1. Browser visits /accounts/signup/
→ urls.py routes to **signup_view()** in views.py

2. GET request: view creates empty **SignUpForm**
→ renders auth/signup.html with the form

3. User fills form and submits (POST)
→ SignUpForm validates: username unique? passwords match? email valid?

4. If valid: **User** created with hashed password
→ **Profile** created (blank) for that User
→ User logged in automatically
→ Redirect to /onboarding/

5. Onboarding form shown, user fills preferences
→ Profile.destinations, travel_months, style, budget, etc. saved
→ onboarding_complete = True
→ Redirect to /matches/

Flow 2: Compatibility Matching

```
1. User visits /matches/
→ match_feed() called

2. services/matching.py: get_ranked_matches(user)
→ Queries all users with onboarding_complete=True
→ Excludes already-acted-on users (via MatchAction table)

3. For each candidate: compute_compatibility(my_prefs, their_prefs)
→ Runs _ordinal_sim, _jaccard_sim, _style_sim
→ Returns weighted sum score (0.0–1.0)

4. Sort by score descending, take top 20
→ Multiply by 100 (for "%" display)
→ Render matches/match_feed.html

5. User clicks "Like" on a card
→ JavaScript POSTs to /matches/<id>/action/
→ match_action() saves MatchAction(actor, target, "like", score)
→ Returns JSON {ok: true}
→ JavaScript fades out and removes the card
```

Flow 3: Real-Time Chat

```
1. User opens group_detail.html (Chat tab is default)
→ Django renders all existing messages server-side

2. JavaScript starts polling loop (every 2 seconds):
    fetch("/groups/<pk>/chat/messages/?since=<lastId>")
→ chat_messages() view returns only NEW messages
→ JavaScript renders them and updates lastId cursor

3. User types message + presses Enter
→ JavaScript POSTs to /groups/<pk>/chat/send/
→ chat_send() creates Message in DB
→ Returns the message as JSON
→ JavaScript immediately renders it in the chat box
→ Other users' polling picks it up within 2 seconds
```

Flow 4: Adding an Expense

```
1. User clicks "Add Expense" → modal opens with ExpenseForm

2. User fills in: description, amount $120, currency USD, split=equal
→ JavaScript submits form via fetch() to /groups/<pk>/expenses/add/

3. expense_create() view:
    - Gets all 3 group member IDs
    - Calculates equal share: $120 ÷ 3 = $40 each

4. Inside transaction.atomic() (all-or-nothing):
    - Creates Expense record ($120)
    - Creates 3 ExpenseAllocation rows ($40 each)
    - Updates BalanceLedger for each member:
      payer: net_balance += (120 - 40) = +$80
      others: net_balance += (0 - 40) = -$40 each

5. Returns {ok: true} → page reloads to show updated list
```

Flow 5: Settlement Plan

```
1. Expenses tab loads → group_detail() reads BalanceLedger
→ Calls compute_group_settlement(group)

2. settlement.py reads net balances from BalanceLedger
→ Splits into creditors (+) and debtors (−)
→ Greedy algorithm matches largest creditor with largest debtor
→ Produces minimum transfer list

3. Template renders: "Carol pays Alice $80" etc.
→ Color coded: red = payer, green = receiver
```

The Django ORM Cheat Sheet

These are the database query patterns used throughout views.py:

CODE	SQL EQUIVALENT
`Profile.objects.get(user=request.user)`	SELECT * WHERE user_id = ? (returns one row)
`TripGroup.objects.filter(owner=user)`	SELECT * WHERE owner_id = ? (returns many)
`TripGroup.objects.exclude(pk__in=ids)`	SELECT * WHERE id NOT IN (...)
`Profile.objects.get_or_create(user=u)`	SELECT; INSERT if not found
`MatchAction.objects.update_or_create(...)`	UPDATE if exists, INSERT if not (upsert)
`.select_related("user")`	JOIN — fetches related row in same query
`.prefetch_related("activities")`	Separate query, cached — avoids N+1 for lists
`expense.delete()`	DELETE FROM expenses WHERE id = ?

💡 **To run the project**

1. `source .venv/bin/activate` — activate virtual environment
2. `python manage.py migrate` — set up the database
3. `python manage.py runserver` — start the server
4. Open `http://127.0.0.1:8000` in your browser